

Pontificia Universidad Javeriana

Facultad de Ingeniería

Ingeniería de Sistemas

Plan de Pruebas

Secure Ticket

Versión del documento: v1.0

Fecha: 18 de mayo de 2026

Autores: *[Juan Diego Arias Durán]*

[Santiago Andrés Mesa Niño]

[Andrés Felipe Manosalva Garzón]

[Juan Camilo Carvajal Rodríguez]

Director: *[Rafael Vicente Páez Méndez, PhD]*

Bogotá, D.C.

2026

Historial de cambios

Versión	Fecha	Sección modificada	Descripción de cambios	Responsable(s)
1.0	[DD/MM/AAAA]	Documento completo	Versión inicial del plan de pruebas de EasyMarket SPL.	[Equipo]

Índice

Historial de cambios	1
1. Introducción	8
1.1. Propósito del documento	8
1.2. Alcance del sistema	8
1.3. Audiencia objetivo	9
1.4. Referencias	10
2. Objetivos del plan de pruebas	10
2.1. Objetivo general	11
2.2. Objetivos específicos	11
2.3. Metas de calidad	12
2.4. Criterios generales de éxito	13
3. Ítems de prueba	14
4. Alcance de las pruebas	18
4.1. Componentes incluidos en el alcance	19
4.2. Flujos funcionales incluidos	20
4.3. Niveles de prueba incluidos	21
4.4. Cobertura parcial por diseño	22
4.5. Elementos fuera del alcance	22
4.6. Criterio de delimitación del alcance	23
5. Estrategia de pruebas	23
5.1. Pruebas unitarias de contratos inteligentes	24
5.2. Pruebas unitarias y de API del backend	24

5.3. Pruebas de integración	25
5.4. Pruebas end-to-end del frontend	26
5.5. Prueba manual guiada de reventa en Testnet	27
5.6. Pruebas del scanner QR	28
5.7. Pruebas de seguridad funcional	28
5.8. Pruebas de carga	29
5.9. Pruebas de humo de despliegue	30
5.10. Pruebas de aceptación de usuario	30
5.11. Trazabilidad de pruebas	31
6. Ambiente y datos de prueba	31
6.1. Ambientes de prueba	32
6.2. Configuración técnica del ambiente	33
6.3. Variables de entorno	34
6.4. Datos de prueba	35
6.5. Estados de ticket considerados	36
6.6. Datos para pruebas de carga	37
6.7. Consideraciones de seguridad sobre datos de prueba	38
6.8. Criterios de preparación del ambiente	38
7. Criterios de entrada y salida	39
7.1. Criterios generales de entrada	39
7.2. Criterios generales de salida	40
7.3. Criterios de entrada por tipo de prueba	41
7.4. Criterios de salida por tipo de prueba	42
7.5. Estados de resultado de prueba	43
7.6. Criterio de suspensión y reanudación	44
8. Casos de prueba	45

8.1. Formato estándar de caso de prueba	45
8.2. Pruebas unitarias	45
8.2.1. Users Service	45
8.2.2. Products Service	46
8.2.3. Orders Service	46
8.2.4. Reviews Service	46
8.3. Pruebas de sistema	47
8.3.1. Procesos seleccionados	47
8.4. Pruebas de carga	48
8.5. Pruebas de aceptación de usuario	48
8.5.1. Formulario de aceptación	48
8.5.2. Usuarios participantes	48
8.6. Pruebas de configurabilidad	49
9. Resultados y análisis	49
9.1. Resumen general de ejecución	49
9.2. Resultados de pruebas unitarias	50
9.3. Resultados de pruebas de sistema	50
9.4. Resultados de pruebas de carga	50
9.5. Resultados de aceptación de usuario	50
9.6. Resultados de configurabilidad	50
9.7. Análisis de defectos	50
10. Riesgos y mitigaciones	51
11. Matriz de trazabilidad	51
12. Conclusiones	52
12.1. Trabajo futuro	52

13. Anexos	52
13.1. Anexo A: Evidencias de pruebas unitarias	52
13.2. Anexo B: Evidencias de pruebas de sistema	52
13.3. Anexo C: Evidencias de pruebas de carga	52
13.4. Anexo D: Formularios de aceptación	52
13.5. Anexo E: Evidencias de configurabilidad	53

Índice de tablas

Índice de figuras

1. Introducción

1.1. Propósito del documento

El propósito de este documento es definir el plan de pruebas de Secure Ticket, estableciendo los ítems de prueba, la estrategia de validación, los tipos de prueba aplicables, los criterios de aceptación, los casos de prueba y la forma de registrar los resultados obtenidos durante la verificación del prototipo.

Este plan sirve como guía para evaluar que los componentes principales del sistema cumplen con los requerimientos funcionales y no funcionales definidos para el proyecto. En particular, busca validar los flujos críticos asociados a la autenticidad, trazabilidad, reventa y verificación de tickets mediante una arquitectura híbrida compuesta por frontend web, backend, base de datos PostgreSQL y contratos inteligentes ejecutados sobre Stellar/Soroban Testnet.

El documento también permite establecer una base objetiva para reportar el estado de calidad del software antes de la entrega final del trabajo de grado, diferenciando entre pruebas automatizadas, pruebas manuales guiadas, pruebas de integración, pruebas de carga, pruebas de aceptación y limitaciones propias del alcance académico del prototipo.

1.2. Alcance del sistema

Secure Ticket es un prototipo académico orientado a soportar la reventa segura de tickets para eventos mediante el uso de contratos inteligentes y tokens no fungibles sobre la red Stellar/Soroban. El sistema permite representar tickets como activos digitales, gestionar su estado, consultar su trazabilidad, habilitar flujos de reventa y verificar tickets mediante un flujo de escaneo QR.

El alcance de este plan cubre las pruebas sobre los siguientes componentes:

- Contratos inteligentes Soroban asociados a la emisión, reventa, cancelación, invalidación y redención de tickets.
- Backend/API responsable de la lógica de negocio, autenticación, autorización por roles, gestión de tickets, scanner QR, validaciones de estado y comunicación con servicios de persistencia.

- Indexador o mecanismo de sincronización entre eventos on-chain y el estado almacenado off-chain en PostgreSQL.
- Base de datos PostgreSQL utilizada para persistir usuarios, eventos, tickets, órdenes, estados y campos asociados a la integración Web3.
- Frontend web utilizado por compradores, vendedores, administradores y personal de verificación.
- Flujos end-to-end del prototipo, incluyendo compra primaria simulada, inventario, scanner QR y reventa en Stellar/Soroban Testnet.
- Pruebas de seguridad funcional enfocadas en roles, tokens, propiedad de tickets y validación de operaciones no autorizadas.
- Pruebas de carga sobre endpoints REST representativos, sin incluir carga masiva sobre transacciones blockchain.
- Pruebas de aceptación con usuarios o stakeholders en un ambiente académico controlado.

Las pruebas blockchain se realizan exclusivamente sobre Stellar/Soroban Testnet. El despliegue en mainnet, el uso de activos con valor real, pagos reales, integraciones con pasarelas de pago productivas, procesos KYC/AML y operación comercial en producción se encuentran fuera del alcance de este plan de pruebas.

Adicionalmente, algunas funcionalidades se validan de acuerdo con el alcance real del prototipo. La compra primaria se evalúa como un flujo simulado/off-chain cuando así corresponde a la implementación actual. El scanner QR se evalúa en modalidad DB-first, mientras que la redención on-chain se valida a nivel de contrato inteligente. La reventa con Freightier se contempla como prueba manual guiada sobre Stellar/Soroban Testnet, debido a la fragilidad técnica de automatizar extensiones de wallet en pruebas end-to-end.

1.3. Audiencia objetivo

Este documento está dirigido a los siguientes perfiles relacionados con el proyecto Secure Ticket:

- Equipo de desarrollo frontend, backend y contratos inteligentes.

- Equipo responsable de pruebas, validación y documentación.
- Director del trabajo de grado.
- Jurados o revisores académicos del proyecto.
- Interesados técnicos que requieran comprender el alcance, estrategia y resultados de las pruebas realizadas.

1.4. Referencias

ID	Referencia
R1	Especificación de Requerimientos de Software de Secure Ticket (SRS).
R2	Documento de Especificación del Diseño o Arquitectura de Software de Secure Ticket (SAD/SDD).
R3	Repositorio oficial del proyecto Secure Ticket.
R4	Documentación oficial de Stellar y Soroban para contratos inteligentes y ejecución sobre Testnet.
R5	Documentación de herramientas de prueba utilizadas en el proyecto: Cargo Test, Jest, Supertest, Playwright, k6 y herramientas asociadas al entorno de desarrollo.
R6	Plan de pruebas de EasyMarket SPL utilizado como referencia estructural para la organización del presente documento.

2. Objetivos del plan de pruebas

El objetivo principal de este plan de pruebas es establecer una estrategia sistemática para verificar y validar el funcionamiento de Secure Ticket, considerando los componentes de software que conforman el prototipo y los flujos críticos definidos para el trabajo de grado. Las pruebas buscan comprobar que el sistema cumple con los requerimientos funcionales y no funcionales asociados a la autenticidad, trazabilidad, reventa y verificación de tickets en un ambiente académico controlado.

Dado que Secure Ticket integra componentes tradicionales de software con contratos inteligentes sobre Stellar/Soroban Testnet, el plan de pruebas contempla una validación en diferentes niveles: contratos, backend, frontend, persistencia, sincronización on-chain/off-chain, pruebas end-to-end, seguridad funcional, carga y aceptación de usuario.

2.1. Objetivo general

Definir y ejecutar una estrategia de pruebas que permita evaluar la calidad funcional, técnica y operativa del prototipo Secure Ticket, verificando que sus componentes principales soportan los flujos de compra primaria simulada, reventa, verificación QR, control de acceso, trazabilidad e interacción con contratos inteligentes sobre Stellar/Soroban Testnet.

2.2. Objetivos específicos

Los objetivos específicos del presente plan de pruebas son:

- Verificar el correcto funcionamiento de los contratos inteligentes Soroban asociados a la emisión, reventa, cancelación, invalidación y redención de tickets.
- Validar que el backend/API implemente correctamente las reglas de negocio relacionadas con autenticación, autorización por roles, gestión de tickets, scanner QR, compra primaria simulada, reventa y restricciones de operación.
- Comprobar que el frontend web permita ejecutar los flujos principales del prototipo desde la perspectiva de los usuarios definidos: comprador, vendedor, administrador y personal de verificación.
- Evaluar la sincronización entre los eventos o estados derivados de la lógica on-chain y la información persistida off-chain en PostgreSQL.
- Validar que el scanner QR permita verificar tickets válidos y rechazar tickets usados, inválidos, inexistentes, vencidos o procesados por usuarios no autorizados.
- Verificar que los mecanismos de seguridad funcional impidan operaciones no autorizadas, incluyendo accesos sin token, tokens inválidos, roles insuficientes, manipulación de solicitudes y operaciones sobre tickets ajenos.
- Evaluar el comportamiento del sistema bajo carga moderada sobre endpoints REST representativos, excluyendo carga masiva sobre transacciones blockchain, Freightier o mainnet.
- Validar la disponibilidad básica del prototipo en el ambiente de despliegue mediante pruebas de humo sobre los servicios principales.

- Obtener una validación académica de aceptación por parte de usuarios o stakeholders mediante tareas representativas del flujo de negocio.
- Registrar los resultados de las pruebas de forma trazable, relacionando cada prueba con los casos de uso, requerimientos o atributos de calidad que busca verificar.

2.3. Metas de calidad

Las metas de calidad que orientan este plan de pruebas son las siguientes:

- **Correctitud funcional:** los flujos principales del prototipo deben ejecutarse de acuerdo con las reglas de negocio definidas para compra, reventa, verificación, invalidación y cancelación de tickets.
- **Integridad:** las operaciones sobre tickets deben preservar estados consistentes, evitando que un ticket usado, invalidado, cancelado o no perteneciente al usuario pueda ser operado indebidamente.
- **Trazabilidad:** el sistema debe permitir relacionar el estado de un ticket con su ciclo de vida, incluyendo propietario, versión, estado de venta, eventos relevantes y vínculo con la lógica blockchain cuando aplique.
- **Seguridad funcional:** las acciones críticas deben estar restringidas según rol, token, propiedad del ticket y validación de wallet cuando corresponda.
- **Confiabilidad operativa:** los flujos críticos deben manejar errores esperados de forma controlada, incluyendo QR inválidos, doble escaneo, tickets inexistentes, tokens inválidos y solicitudes manipuladas.
- **Rendimiento básico:** los endpoints REST representativos deben responder de manera estable bajo una carga moderada, dentro de umbrales acordes al alcance académico del prototipo.
- **Usabilidad básica:** los usuarios deben poder completar tareas representativas del sistema, como comprar un ticket, consultar inventario, listar una reventa o escanear un QR, con una comprensión razonable del estado y resultado de cada operación.

2.4. Criterios generales de éxito

Los criterios generales de éxito permiten establecer las condiciones mínimas bajo las cuales el proceso de pruebas se considera satisfactorio para el alcance académico de Secure Ticket. Estos criterios no implican operación productiva ni despliegue en mainnet, sino validación del prototipo en los ambientes definidos para el proyecto.

Categoría	Criterio de éxito	Tipo de validación
Contratos inteligentes	Las pruebas de contratos Soroban deben ejecutarse sin fallos bloqueantes, cubriendo emisión, reventa, cancelación, invalidación, redención y casos negativos relevantes.	Automatizada
Backend/API	Las pruebas unitarias y de API deben validar reglas de negocio, autenticación, autorización, scanner QR, operaciones sobre tickets y manejo de errores esperados.	Automatizada
Frontend/E2E	Los flujos principales del prototipo deben poder ejecutarse desde la interfaz web, incluyendo compra primaria simulada, inventario y scanner QR.	Automatizada / Manual
Reventa en Testnet	La reventa debe validarse mediante una prueba manual guiada sobre Stellar/Soroban Testnet, usando Freightier y verificando cambio de propietario, trazabilidad y actualización del estado del ticket.	Manual guiada
Scanner QR	El scanner debe aceptar tickets válidos y rechazar tickets usados, inválidos, inexistentes, expirados o procesados por usuarios no autorizados.	Automatizada / E2E
Redención on-chain	La redención on-chain debe estar validada a nivel de contrato inteligente, aunque el scanner operativo del prototipo funcione bajo un enfoque DB-first.	Automatizada

Sincronización on-chain/off-chain	Las pruebas de integración deben validar que los eventos o estados relevantes se reflejan correctamente en PostgreSQL, sin duplicaciones ni inconsistencias visibles.	Integración
Seguridad funcional	El sistema debe rechazar accesos sin token, tokens inválidos o vencidos, roles insuficientes, operaciones sobre tickets ajenos y solicitudes manipuladas.	Automatizada
Carga y rendimiento	Los escenarios de carga deben cubrir lectura pública, usuarios autenticados y operaciones críticas controladas, manteniendo tasas de error y tiempos de respuesta dentro de los umbrales definidos para el prototipo.	Automatizada
Despliegue	El smoke test debe confirmar que el frontend, backend, autenticación, consulta de eventos, inventario y scanner funcionan en el ambiente desplegado.	Manual guiada
Aceptación de usuario	Las pruebas de aceptación deben permitir evaluar si usuarios o stakeholders pueden completar tareas representativas y comprender los estados principales del ticket.	Manual / Formulario
Trazabilidad	Cada prueba crítica debe estar asociada a un requerimiento, caso de uso o atributo de calidad mediante la matriz de trazabilidad.	Documental
Limitaciones	Las limitaciones del prototipo deben quedar documentadas explícitamente, incluyendo exclusión de mainnet, pagos reales, carga blockchain masiva y redención on-chain desde el scanner operativo.	Documental

3. Ítems de prueba

Los ítems de prueba corresponden a los componentes, servicios, flujos e integraciones de Secure Ticket que serán evaluados durante el proceso de verificación y validación. Estos

elementos fueron seleccionados por su relevancia dentro del funcionamiento del prototipo y por su relación directa con los objetivos del sistema: autenticidad, trazabilidad, reventa y verificación de tickets.

Dado que Secure Ticket utiliza una arquitectura híbrida, los ítems de prueba incluyen tanto componentes tradicionales de software, como frontend, backend y base de datos, como componentes asociados a la lógica blockchain, tales como contratos inteligentes Soroban e integración con Stellar/Soroban Testnet.

Nombre	Tipo	Descripción	Objetivo de prueba
Contrato <i>Fabrica-Boletos</i>	Contrato inteligente	Contrato Soroban encargado de gestionar o desplegar contratos asociados a eventos, según el modelo de arquitectura definido para el sistema.	Verificar que la creación y administración de contratos de evento se realice correctamente bajo las reglas definidas.
Contrato <i>Contra-toEvento</i>	Contrato inteligente	Contrato Soroban que contiene la lógica principal relacionada con emisión, reventa, cancelación, invalidación y redención de tickets.	Validar que las operaciones críticas sobre tickets se ejecuten correctamente y que los casos inválidos sean rechazados.
Contrato de representación NFT del ticket	Contrato inteligente / Activo digital	Componente asociado a la representación del ticket como activo digital único, incluyendo identificadores, versión, propietario y estado.	Verificar la unicidad, trazabilidad y relación del ticket con su propietario y ciclo de vida.
Backend/API	Servicio backend	Servicio responsable de exponer endpoints REST, aplicar reglas de negocio, autenticar usuarios, validar roles, procesar operaciones sobre tickets y coordinar la comunicación con persistencia e integraciones.	Verificar el correcto funcionamiento de las reglas de negocio, autorización, validaciones de estado y respuestas ante errores esperados.

Indexador Soroban	Servicio de sincronización	Componente encargado de procesar eventos o estados derivados de la lógica on-chain y reflejarlos en la base de datos off-chain.	Validar la sincronización entre eventos blockchain y PostgreSQL, evitando duplicaciones o inconsistencias visibles.
Base de datos PostgreSQL	Persistencia	Base de datos relacional utilizada para almacenar usuarios, eventos, tickets, órdenes, estados, check-ins y campos asociados a la integración Web3.	Verificar la consistencia de los datos persistidos después de operaciones de compra, reventa, verificación, cancelación e invalidación.
Frontend web	Interfaz gráfica	Aplicación web utilizada por compradores, vendedores, administradores y personal de verificación para interactuar con el sistema.	Validar que los usuarios puedan ejecutar los flujos principales del prototipo desde la interfaz, incluyendo compra primaria simulada, inventario y scanner QR.
Scanner QR	Flujo funcional / Interfaz de verificación	Funcionalidad utilizada por personal autorizado para verificar tickets mediante códigos QR. En el prototipo operativo, el scanner funciona bajo un enfoque DB-first.	Comprobar que el scanner acepte tickets válidos y rechace tickets usados, inválidos, inexistentes, expirados o procesados por usuarios no autorizados.
Módulo de autenticación y autorización	Servicio de seguridad	Mecanismo encargado de validar identidad, tokens, roles y permisos para acceder a operaciones protegidas del sistema.	Verificar que únicamente los usuarios autorizados puedan ejecutar acciones críticas según su rol y propiedad sobre los tickets.

Flujo de compra primaria simulada	Flujo funcional	Proceso mediante el cual un usuario adquiere un ticket dentro del prototipo, generando una orden y reflejando el ticket en su inventario.	Validar que la compra primaria simulada funcione correctamente sin afirmar operación con pagos reales ni transacción on-chain cuando no corresponda.
Flujo de reventa en Stellar/Soroban Testnet	Flujo funcional / Integración blockchain	Proceso mediante el cual un propietario lista un ticket para reventa y otro usuario lo adquiere utilizando Freighters sobre Stellar/Soroban Testnet.	Validar el flujo central de reventa, verificando cambio de propietario, trazabilidad, actualización de estado y relación con la lógica blockchain del prototipo.
Integración con Freighters Wallet	Integración externa	Extensión de wallet utilizada para firmar operaciones blockchain por parte de usuarios en Stellar/Soroban Testnet.	Verificar manualmente que los flujos que requieren firma de usuario puedan ejecutarse en Testnet sin usar mainnet ni activos reales.
Integración con Stellar/Soroban Testnet	Integración externa	Red de pruebas utilizada para desplegar o ejecutar operaciones blockchain dentro del alcance académico del sistema.	Validar operaciones blockchain en un ambiente de prueba, excluyendo mainnet, activos con valor real y operación productiva.
Pruebas de carga sobre API REST	Escenario de rendimiento	Conjunto de escenarios k6 o equivalentes orientados a evaluar endpoints de lectura pública, usuarios autenticados y operaciones críticas controladas.	Evaluar estabilidad y tiempos de respuesta del backend bajo carga moderada, sin ejecutar carga masiva sobre Freighters o transacciones Soroban reales.

Pruebas de aceptación de usuario	Validación con usuarios	Actividades guiadas para que usuarios o stakeholders ejecuten tareas representativas y evalúen claridad, comprensión y confianza del sistema.	Obtener retroalimentación sobre la usabilidad básica y comprensión de los flujos principales del prototipo.
Smoke test de despliegue	Validación de ambiente	Prueba de humo orientada a verificar que el frontend, backend y flujos mínimos funcionen en el ambiente desplegado.	Confirmar la disponibilidad básica del prototipo fuera del ambiente local de desarrollo.
Matriz de trazabilidad	Artefacto documental	Relación entre pruebas, requerimientos, casos de uso y atributos de calidad definidos para el sistema.	Asegurar que las pruebas ejecutadas se encuentren asociadas a los elementos funcionales y no funcionales que buscan validar.

Los ítems de prueba descritos en esta sección delimitan el alcance de la estrategia de validación. Aquellos elementos que involucren mainnet, pagos reales, activos con valor económico, procesos KYC/AML o integraciones productivas quedan excluidos del presente plan por encontrarse fuera del alcance académico del prototipo.

4. Alcance de las pruebas

El alcance de las pruebas de Secure Ticket comprende la verificación y validación de los componentes principales del prototipo académico, así como de los flujos funcionales que soportan la autenticidad, trazabilidad, reventa y verificación de tickets. Las pruebas se enfocan en evaluar el comportamiento del sistema en ambientes controlados de desarrollo, integración, despliegue académico y Stellar/Soroban Testnet.

Este plan no busca certificar el sistema para operación productiva, sino demostrar que el prototipo implementa correctamente los mecanismos definidos dentro del alcance del trabajo de grado.

4.1. Componentes incluidos en el alcance

Las pruebas cubren los siguientes componentes del sistema:

- **Contratos inteligentes Soroban:** se validan las funciones relacionadas con emisión, compra, reventa, cancelación, invalidación, redención, permisos de verificadores y casos negativos sobre tickets.
- **Backend/API:** se evalúan los endpoints REST, reglas de negocio, autenticación, autorización por roles, validaciones de estado, scanner QR, compra primaria simulada, reventa, cancelación e invalidación según el alcance implementado.
- **Frontend web:** se prueban los flujos principales de usuario disponibles en la interfaz, incluyendo inicio de sesión, consulta de eventos, compra primaria simulada, inventario de tickets y scanner QR.
- **Base de datos PostgreSQL:** se valida la persistencia y consistencia de entidades como usuarios, eventos, tickets, órdenes, estados, check-ins y campos asociados a la integración Web3.
- **Indexador o sincronización on-chain/off-chain:** se prueban los mecanismos encargados de reflejar eventos o estados asociados a la lógica blockchain en la base de datos off-chain.
- **Integración con Stellar/Soroban Testnet:** se valida la lógica blockchain en red de pruebas, principalmente para flujos relacionados con contratos inteligentes y reventa manual guiada.
- **Integración con Freightier Wallet:** se contempla dentro del flujo manual guiado de reventa en Testnet, debido a que la automatización de extensiones de wallet puede ser frágil y poco estable.
- **Pruebas de carga sobre API REST:** se incluyen escenarios sobre lectura pública, usuarios autenticados y operaciones críticas controladas.
- **Pruebas de aceptación:** se incluye la validación académica con usuarios o stakeholders sobre tareas representativas del sistema.
- **Smoke test de despliegue:** se valida que los servicios principales del prototipo funcionen en el ambiente desplegado.

4.2. Flujos funcionales incluidos

Los flujos funcionales cubiertos por este plan de pruebas son los siguientes:

Flujo	Descripción	Tipo de prueba asociada
Compra primaria simulada	Flujo mediante el cual un usuario selecciona un evento, realiza un checkout simulado y recibe un ticket en su inventario.	E2E / Backend / Frontend
Reventa en Testnet	Flujo mediante el cual un usuario propietario lista un ticket para reventa y otro usuario lo adquiere usando Freighter sobre Stellar/Soroban Testnet.	Manual guiada / Blockchain
Scanner QR	Flujo mediante el cual un usuario autorizado verifica un ticket mediante QR, registrando el uso del ticket y rechazando casos inválidos.	API / E2E / Seguridad
Redención on-chain	Validación de la función de redención del ticket en el contrato inteligente Soroban.	Contrato inteligente
Invalidación de boleto	Validación de que un ticket invalidado no pueda listarse, comprarse, escanearse o redimirse según el nivel implementado.	Contrato / Backend / API
Cancelación de reventa	Validación de que un ticket listado pueda retirarse del mercado secundario y no pueda ser comprado posteriormente.	Contrato / Backend / API
Autenticación y autorización	Validación de tokens, roles, permisos, propiedad de tickets y operaciones no autorizadas.	Seguridad / API
Sincronización on-chain/off-chain	Validación de que los eventos o estados relevantes se reflejen correctamente en PostgreSQL.	Integración

Carga sobre API REST	Evaluación del comportamiento del backend bajo carga moderada en endpoints representativos.	Rendimiento
Aceptación de usuario	Validación de que usuarios o stakeholders puedan completar tareas representativas y comprender los estados principales del sistema.	UAT / Formulario

4.3. Niveles de prueba incluidos

El plan contempla los siguientes niveles de prueba:

- **Pruebas unitarias:** orientadas a validar componentes aislados, principalmente contratos inteligentes y lógica de backend.
- **Pruebas de integración:** enfocadas en la interacción entre backend, base de datos, indexador y eventos asociados a la lógica on-chain.
- **Pruebas de API:** dirigidas a verificar endpoints, códigos de respuesta, validaciones, manejo de errores y reglas de autorización.
- **Pruebas end-to-end:** orientadas a validar flujos principales desde la perspectiva del usuario a través de la interfaz web.
- **Pruebas manuales guiadas:** utilizadas para flujos donde la automatización no es conveniente o estable, especialmente la reventa con Freightier sobre Stellar/Soroban Testnet.
- **Pruebas de carga:** enfocadas en medir el comportamiento de endpoints REST bajo carga moderada.
- **Pruebas de aceptación:** orientadas a obtener retroalimentación de usuarios o stakeholders sobre tareas representativas del prototipo.
- **Pruebas documentales:** utilizadas para mantener trazabilidad entre requerimientos, casos de uso, atributos de calidad y pruebas ejecutadas.

4.4. Cobertura parcial por diseño

Algunas funcionalidades se consideran parcialmente cubiertas debido al alcance real del prototipo y a decisiones arquitectónicas del sistema. Estas condiciones se documentan explícitamente para evitar afirmar capacidades que no forman parte de la implementación actual.

Elemento	Cobertura actual	Justificación
Scanner QR on-chain	El scanner operativo se valida bajo un enfoque DB-first; la redención on-chain se prueba a nivel de contrato.	La integración directa de redención on-chain en el scanner no hace parte del flujo operativo actual del prototipo.
Invalidación administrativa por API	La invalidación se cubre mediante contrato, reglas backend y validaciones en flujos de scanner, listado y compra.	No existe un endpoint administrativo completo de invalidación en la versión actual del prototipo.
Reventa con Freightier	La reventa se valida mediante una prueba manual guiada sobre Stellar/Soroban Testnet.	Automatizar Freightier puede generar pruebas frágiles y poco mantenibles.
Compra primaria	La compra primaria se valida como flujo simulado/off-chain del prototipo.	El uso de pagos reales o transacciones productivas queda fuera del alcance académico.
Carga blockchain	No se ejecuta carga masiva sobre Freightier ni transacciones Soroban reales.	Las pruebas de carga se limitan a endpoints REST representativos para evitar resultados poco representativos o riesgos innecesarios sobre Testnet.

4.5. Elementos fuera del alcance

Quedan fuera del alcance de este plan de pruebas los siguientes elementos:

- Despliegue o validación sobre Stellar Mainnet.
- Uso de activos con valor real.
- Integración con pagos reales o pasarelas de pago productivas.
- Validaciones comerciales, legales o financieras asociadas a operación productiva.
- Procesos KYC/AML.
- Pruebas de penetración formales o auditorías externas de seguridad.
- Pruebas de carga masiva sobre transacciones blockchain reales.
- Automatización completa de flujos con Freightier Wallet.
- Pruebas sobre aplicaciones móviles nativas.
- Certificación de disponibilidad, escalabilidad o tolerancia a fallos para producción.

4.6. Criterio de delimitación del alcance

Una prueba se considera dentro del alcance cuando evalúa una funcionalidad implementada en el prototipo, puede ejecutarse en ambiente local, staging, despliegue académico o Stellar/Soroban Testnet, y contribuye a validar uno de los objetivos principales del sistema: autenticidad, trazabilidad, reventa o verificación de tickets.

Por el contrario, una prueba se considera fuera del alcance cuando requiere operación en mainnet, activos reales, pagos productivos, infraestructura comercial, validaciones legales especializadas o capacidades no implementadas en la versión actual del sistema.

5. Estrategia de pruebas

La estrategia de pruebas de Secure Ticket se define a partir de una combinación de pruebas automatizadas, pruebas de integración, pruebas end-to-end, pruebas manuales guiadas, pruebas de carga y pruebas de aceptación. Esta combinación responde a la naturaleza híbrida del sistema, el cual integra frontend web, backend/API, base de datos PostgreSQL, contratos inteligentes Soroban y flujos sobre Stellar/Soroban Testnet.

El enfoque general consiste en validar primero los componentes de forma aislada, posteriormente sus integraciones principales y finalmente los flujos funcionales que representan

el comportamiento esperado del prototipo desde la perspectiva del usuario. De esta forma, las pruebas se organizan en niveles progresivos: contratos, backend, integración, frontend, flujos end-to-end, carga, despliegue y aceptación.

Todas las pruebas relacionadas con blockchain se ejecutan en entornos locales de contrato o en Stellar/Soroban Testnet. No se contempla el uso de mainnet, activos reales ni pagos productivos, debido a que estos elementos se encuentran fuera del alcance académico del proyecto.

5.1. Pruebas unitarias de contratos inteligentes

Las pruebas unitarias de contratos inteligentes tienen como objetivo validar la lógica on-chain implementada en Soroban. Estas pruebas permiten verificar el comportamiento de las funciones críticas de los contratos sin depender de la interfaz gráfica ni de flujos externos del sistema.

En este nivel se validan operaciones como emisión de tickets, reventa, cancelación, invalidación, redención, asignación de verificadores autorizados, control de estados y rechazo de casos inválidos.

- **Herramientas:** Cargo Test, entorno de pruebas de Rust y Soroban SDK.
- **Ambiente:** ambiente local de desarrollo para contratos inteligentes.
- **Ítems evaluados:** contratos *FabricaBoletos*, *ContratoEvento* y componentes asociados a la representación del ticket.
- **Criterios de completitud:**
 - Ejecución exitosa de las pruebas de contrato.
 - Cobertura de emisión, reventa, cancelación, invalidación y redención.
 - Inclusión de casos negativos para usuarios no autorizados, tickets inexistentes, tickets usados o tickets inválidos.
 - Validación de que los estados del ticket impidan operaciones no permitidas.

5.2. Pruebas unitarias y de API del backend

Las pruebas del backend permiten validar la lógica de negocio expuesta por la API, incluyendo autenticación, autorización, reglas de operación sobre tickets, scanner QR,

cancelación, validaciones de estado y manejo de errores esperados.

Estas pruebas verifican que el backend aplique restricciones de seguridad y reglas de negocio de forma independiente al frontend, evitando que operaciones críticas dependan únicamente de validaciones visuales o del cliente.

- **Herramientas:** framework de pruebas del backend, Supertest o herramientas equivalentes utilizadas en el repositorio.
- **Ambiente:** ambiente local de desarrollo e integración con base de datos de prueba.
- **Ítems evaluados:** endpoints REST, servicios de autenticación, autorización, scanner QR, operaciones sobre tickets, reglas de reventa e invalidación según el alcance implementado.
- **Criterios de completitud:**
 - Ejecución exitosa de las pruebas unitarias y de API.
 - Validación de respuestas esperadas ante tokens inválidos, expirados o ausentes.
 - Validación de restricciones por roles *CUSTOMER*, *STAFF* y *ADMIN*.
 - Rechazo de operaciones sobre tickets ajenos, usados, inexistentes o inválidos.
 - Manejo controlado de errores de negocio y solicitudes manipuladas.

5.3. Pruebas de integración

Las pruebas de integración tienen como propósito validar la interacción entre componentes del sistema que no pueden evaluarse de forma completamente aislada. En Secure Ticket, estas pruebas se enfocan principalmente en la relación entre backend, base de datos PostgreSQL, indexador y eventos asociados a la lógica on-chain.

El objetivo es comprobar que los cambios derivados de operaciones sobre tickets se reflejen de forma consistente en la persistencia off-chain, evitando duplicaciones, pérdida de eventos o estados inconsistentes.

- **Herramientas:** framework de pruebas del backend, base de datos de prueba, fixtures o eventos normalizados.
- **Ambiente:** ambiente local de integración con PostgreSQL.

- **Ítems evaluados:** indexador Soroban, sincronización on-chain/off-chain, persistencia de tickets, estados y campos Web3.
- **Criterios de completitud:**
 - Procesamiento correcto de eventos de boleto creado, listado, cancelado, revendido, redimido e invalidado.
 - Actualización correcta de campos como propietario, versión, estado de venta y precio de reventa.
 - Manejo controlado de eventos desconocidos o mal formados.
 - Idempotencia ante eventos duplicados.
 - Ausencia de inconsistencias visibles entre el estado esperado y el estado persistido.

5.4. Pruebas end-to-end del frontend

Las pruebas end-to-end permiten validar flujos completos desde la perspectiva del usuario mediante la interfaz web. En este plan se priorizan los flujos principales que representan el uso esperado del prototipo: inicio de sesión, consulta de eventos, compra primaria simulada, inventario y scanner QR.

Estas pruebas no buscan cubrir exhaustivamente todos los estados visuales de la aplicación, sino comprobar que los usuarios puedan completar tareas funcionales críticas utilizando la interfaz.

- **Herramientas:** Playwright o herramienta E2E equivalente configurada en el frontend.
- **Ambiente:** ambiente local o despliegue académico con frontend y backend disponibles.
- **Ítems evaluados:** frontend web, navegación, compra primaria simulada, inventario, scanner QR y mensajes de error.
- **Criterios de completitud:**
 - El usuario puede iniciar sesión y navegar por eventos.

- El usuario puede ejecutar una compra primaria simulada y visualizar el ticket en inventario.
- El scanner QR puede mostrar resultados de éxito y rechazo según el caso evaluado.
- Los usuarios no autorizados son bloqueados en flujos restringidos.
- Los errores principales se presentan de forma comprensible para el usuario.

5.5. Prueba manual guiada de reventa en Testnet

La reventa con Freightier Wallet se define como una prueba manual guiada debido a que la automatización de extensiones de wallet puede producir pruebas frágiles y difíciles de mantener. Esta prueba permite validar el flujo central del sistema sobre Stellar/Soroban Testnet sin involucrar mainnet ni activos reales.

El flujo contempla que un usuario propietario liste un ticket para reventa y que un segundo usuario lo compre mediante una operación firmada con Freightier. Se verifican el cambio de propietario, la actualización de estado, la trazabilidad del ticket y la consistencia con PostgreSQL.

- **Herramientas:** navegador web, Freightier Wallet, frontend del sistema, backend/API, PostgreSQL y Stellar/Soroban Testnet.
- **Ambiente:** ambiente local o despliegue académico configurado contra Stellar/Soroban Testnet.
- **Ítems evaluados:** flujo de reventa, integración con Freightier, contrato en Testnet, marketplace, inventario y persistencia.
- **Criterios de completitud:**
 - Usuario propietario puede listar un ticket para reventa.
 - Un segundo usuario puede comprar el ticket listado utilizando Freightier.
 - La operación se realiza sobre Stellar/Soroban Testnet.
 - Se verifica cambio de propietario y actualización del inventario.
 - Se valida que el ticket no permanezca disponible indebidamente para el propietario anterior.
 - Se prueban casos negativos como compra del propio ticket, listado de ticket ajeno o rechazo de firma.

5.6. Pruebas del scanner QR

Las pruebas del scanner QR se enfocan en validar el flujo de verificación de tickets desde la perspectiva operativa del prototipo. El scanner funciona bajo un enfoque DB-first, por lo cual la validación de uso del ticket y registro de check-in se realiza principalmente contra la base de datos del sistema.

La redención on-chain se evalúa de forma separada mediante pruebas de contrato inteligente, sin afirmar que el scanner operativo ejecute redención directamente sobre Soroban.

- **Herramientas:** pruebas de API, pruebas E2E y frontend del scanner.
- **Ambiente:** ambiente local o despliegue académico.
- **Ítems evaluados:** scanner QR, control de acceso por rol, check-ins, estados de ticket y manejo de QR inválidos.
- **Criterios de completitud:**
 - El scanner acepta tickets válidos con usuarios autorizados.
 - El sistema rechaza QR falsos, expirados, stale, inexistentes, usados o asociados a tickets inválidos.
 - Se impide el doble escaneo de un mismo ticket.
 - Se restringe el acceso al scanner para usuarios no autorizados.
 - El flujo crea o actualiza los registros de check-in de forma consistente.

5.7. Pruebas de seguridad funcional

Las pruebas de seguridad funcional buscan verificar que las acciones críticas del sistema estén protegidas mediante autenticación, autorización por roles, validación de propiedad del ticket y validación de solicitudes manipuladas.

Estas pruebas no corresponden a una auditoría de seguridad formal ni a un pentest, sino a la validación de controles funcionales esperados para el prototipo.

- **Herramientas:** pruebas de API, Supertest o herramienta equivalente.
- **Ambiente:** ambiente local de pruebas.

- **Ítems evaluados:** autenticación JWT, roles, endpoints protegidos, propiedad de tickets, wallet asociada y validaciones de solicitud.
- **Criterios de completitud:**
 - Rechazo de solicitudes sin token, con token inválido o token expirado.
 - Rechazo de usuarios con rol insuficiente en endpoints protegidos.
 - Rechazo de operaciones sobre tickets ajenos.
 - Rechazo de manipulación de precio, estado o identificadores desde el request.
 - Validación de que las restricciones se apliquen en backend y no únicamente en frontend.

5.8. Pruebas de carga

Las pruebas de carga buscan evaluar el comportamiento del backend/API bajo una carga moderada y representativa del prototipo. Estas pruebas se limitan a endpoints REST y no incluyen carga masiva sobre Freightier, mainnet, pagos reales ni transacciones Soroban reales.

La estrategia contempla tres escenarios: lectura pública, usuarios autenticados y operaciones críticas controladas.

- **Herramientas:** k6 o herramienta equivalente.
- **Ambiente:** ambiente local o despliegue académico.
- **Ítems evaluados:** endpoints de salud, eventos, marketplace, login, inventario, scanner y checkout simulado cuando aplique.
- **Criterios de completitud:**
 - Ejecución de un escenario de lectura pública con carga moderada.
 - Ejecución de un escenario de usuarios autenticados.
 - Ejecución de un escenario de operaciones críticas controladas.
 - Tasa de error menor o igual al umbral definido para el prototipo.
 - Tiempos de respuesta p95 dentro de los umbrales establecidos para cada escenario.
 - Separación explícita entre pruebas REST y operaciones blockchain.

5.9. Pruebas de humo de despliegue

Las pruebas de humo de despliegue permiten verificar que el sistema funcione de manera básica en el ambiente desplegado. Su propósito no es realizar una validación exhaustiva, sino confirmar que los servicios principales están disponibles y que los flujos mínimos pueden ejecutarse.

- **Herramientas:** navegador web, endpoints de salud, frontend desplegado y backend desplegado.
- **Ambiente:** ambiente de despliegue académico.
- **Ítems evaluados:** frontend, backend, autenticación, eventos, marketplace, inventario y scanner.
- **Criterios de completitud:**
 - El frontend carga correctamente.
 - El backend responde en el endpoint de salud.
 - El usuario puede iniciar sesión.
 - Los eventos y marketplace se consultan correctamente.
 - El inventario y scanner funcionan según el rol correspondiente.
 - No se presentan errores críticos visibles en consola o backend durante el flujo mínimo.

5.10. Pruebas de aceptación de usuario

Las pruebas de aceptación de usuario permiten obtener una validación académica controlada sobre la comprensión y ejecución de tareas representativas del sistema. Estas pruebas se orientan a usuarios o stakeholders que interactúan con el prototipo desde roles como comprador, vendedor, staff o administrador.

- **Herramientas:** formulario de aceptación, guía de tareas y ambiente del prototipo.
- **Ambiente:** ambiente local, staging o despliegue académico.
- **Ítems evaluados:** claridad del flujo, comprensión del estado del ticket, confianza percibida sobre autenticidad y capacidad de completar tareas.

- **Criterios de completitud:**

- Participación de usuarios o stakeholders en tareas representativas.
- Registro de resultados mediante formulario con escala de valoración.
- Identificación de errores, confusiones o sugerencias.
- Consolidación de resultados por rol y tarea.
- Análisis de la aceptación general del prototipo dentro del alcance académico.

5.11. Trazabilidad de pruebas

La trazabilidad permite relacionar cada prueba con los requerimientos, casos de uso o atributos de calidad que busca validar. Esta actividad permite justificar la cobertura del plan de pruebas y facilita identificar funcionalidades cubiertas, parcialmente cubiertas o fuera del alcance.

- **Herramientas:** matriz de trazabilidad.
- **Ambiente:** artefacto documental del proyecto.
- **Ítems evaluados:** relación entre pruebas, requerimientos, casos de uso, atributos de calidad y estado de ejecución.
- **Criterios de completitud:**
 - Cada prueba crítica cuenta con identificador único.
 - Cada prueba se asocia con al menos un requerimiento, caso de uso o atributo de calidad.
 - La matriz diferencia estado de implementación y estado de ejecución.
 - Las coberturas parciales y exclusiones por alcance quedan documentadas.

6. Ambiente y datos de prueba

Esta sección describe los ambientes, herramientas, configuraciones y datos utilizados para ejecutar las pruebas de Secure Ticket. Dado que el sistema corresponde a un prototipo académico, las pruebas se realizan en ambientes controlados y no contemplan operación sobre mainnet, pagos reales ni activos con valor económico.

El ambiente de pruebas se organiza en cuatro niveles principales: ambiente local de desarrollo, ambiente de integración, ambiente de despliegue académico y ambiente blockchain de pruebas sobre Stellar/Soroban Testnet. Esta separación permite validar los componentes de manera aislada y posteriormente verificar los flujos principales del sistema de forma integrada.

6.1. Ambientes de prueba

Los ambientes considerados para la ejecución de pruebas son los siguientes:

Ambiente	Descripción	Uso dentro del plan de pruebas
Ambiente local de desarrollo	Entorno ejecutado en las máquinas del equipo de desarrollo, con frontend, backend, base de datos y contratos configurados localmente o contra servicios de prueba.	Ejecución de pruebas unitarias, pruebas de API, pruebas de integración, pruebas E2E locales y validaciones iniciales de flujos.
Ambiente de integración	Entorno utilizado para validar la interacción entre backend, base de datos, indexador y eventos normalizados asociados a la lógica on-chain.	Ejecución de pruebas de integración, sincronización on-chain/off-chain y validación de persistencia.
Ambiente de despliegue académico	Entorno desplegado para validar que el prototipo pueda ejecutarse fuera del ambiente local, incluyendo frontend, backend y servicios asociados.	Ejecución de smoke test de despliegue, pruebas de aceptación y validación básica de disponibilidad.
Stellar/Soroban Testnet	Red de pruebas utilizada para validar operaciones blockchain sin activos reales ni exposición a mainnet.	Ejecución de pruebas relacionadas con contratos inteligentes, reventa manual guiada con Freightier y validación de transacciones de prueba.

6.2. Configuración técnica del ambiente

La configuración técnica contempla los componentes principales requeridos para ejecutar el prototipo y sus pruebas.

Componente	Tecnología / herramienta	Propósito en pruebas
Frontend web	React, Vite, TypeScript y Playwright	Ejecutar la interfaz del usuario y validar flujos end-to-end del prototipo.
Backend/API	Node.js, Express, TypeScript, framework de pruebas del backend y Supertest	Ejecutar reglas de negocio, endpoints REST, pruebas unitarias, pruebas de API y validaciones de seguridad funcional.
Contratos inteligentes	Rust, Soroban SDK y Cargo Test	Validar la lógica on-chain de emisión, reventa, cancelación, invalidación y redención de tickets.
Base de datos	PostgreSQL	Persistir usuarios, eventos, tickets, órdenes, check-ins y campos asociados a la integración Web3.
ORM / migraciones	Prisma o herramienta equivalente definida en el repositorio	Gestionar el esquema de base de datos y validar el estado de migraciones.
Indexador	Servicio de sincronización implementado en backend	Procesar eventos o estados asociados a la lógica on-chain y reflejarlos en PostgreSQL.
Wallet	Freighter Wallet	Firmar operaciones de usuario en Stellar/Soroban Testnet durante la prueba manual guiada de reventa.
Red blockchain	Stellar/Soroban Testnet	Ejecutar operaciones blockchain de prueba sin usar mainnet ni activos reales.

Pruebas de carga	k6	Ejecutar escenarios de carga sobre endpoints REST representativos.
Pruebas de aceptación	Formulario de aceptación y guía de tareas	Recoger retroalimentación de usuarios o stakeholders sobre los flujos principales del prototipo.

6.3. Variables de entorno

Para ejecutar las pruebas se requiere configurar variables de entorno asociadas al frontend, backend, base de datos, autenticación, contratos y red Stellar/Soroban Testnet. Los valores sensibles no deben incluirse en este documento ni en el repositorio público; únicamente se documentan los nombres de variables y su propósito.

Variable	Propósito	Componente
DATABASE_URL	Cadena de conexión a PostgreSQL para ejecución del backend y pruebas de integración.	Backend / BD
JWT_SECRET	Secreto utilizado para firmar o validar tokens de autenticación en ambiente de prueba.	Backend
SOROBAN_RPC_URL	URL del endpoint RPC de Stellar/Soroban Testnet.	Backend / Blockchain
STELLAR_NETWORK	Identificador de la red Stellar utilizada. Para pruebas debe corresponder a Testnet.	Backend / Blockchain
CONTRACT_ADDRESS	Dirección del contrato utilizado en pruebas o flujos de integración cuando aplique.	Backend / Blockchain
FACTORY_CONTRACT_ADDRESS	Dirección del contrato Factory cuando el flujo lo requiera.	Backend / Blockchain
FRONTEND_URL	URL del frontend en ambiente local o desplegado.	Frontend / E2E

BACKEND_URL	URL del backend/API utilizada por pruebas E2E, smoke test o carga.	Backend / E2E
BASE_URL	URL base usada por los scripts de carga k6.	Carga
E2E_REAL	Bandera para habilitar pruebas E2E contra ambiente real cuando aplique.	Frontend / E2E

Los valores concretos de estas variables deben definirse en archivos locales de configuración, por ejemplo `.env`, `.env.test` o `.env.example`, según la estructura del repositorio. Ningún secreto real debe incluirse dentro del documento de pruebas.

6.4. Datos de prueba

Los datos de prueba se definen para representar los actores, eventos, tickets y estados necesarios para ejecutar los escenarios funcionales del sistema. Estos datos pueden provenir de seed data, fixtures, registros creados durante la prueba o configuraciones manuales del ambiente.

Tipo de dato	Descripción	Uso en pruebas
Usuarios compradores	Cuentas con rol <i>CUSTOMER</i> utilizadas para compra primaria simulada, inventario y compra de reventa.	Pruebas E2E, API, UAT y reventa manual guiada.
Usuario vendedor	Cuenta con rol <i>CUSTOMER</i> que posee tickets disponibles para listar en reventa.	Prueba manual guiada de reventa y validación de marketplace.
Usuario staff	Cuenta con rol <i>STAFF</i> autorizada para operar el scanner QR.	Pruebas de scanner, seguridad por roles y smoke test.
Usuario administrador	Cuenta con rol <i>ADMIN</i> utilizada para validar operaciones administrativas permitidas.	Pruebas de seguridad, scanner, smoke test y flujos administrativos.
Wallets de prueba	Cuentas Freighter o llaves de Stellar Testnet asociadas a usuarios de prueba.	Reventa manual guiada, firma de transacciones y validación de propietario.

Eventos de prueba	Eventos registrados en el sistema con tickets disponibles o asociados a contratos.	Compra primaria, inventario, marketplace y reventa.
Tickets disponibles	Tickets activos y operables asociados a eventos de prueba.	Compra, reventa, inventario y scanner.
Tickets usados	Tickets marcados como usados o con check-in registrado.	Validación de rechazo de doble escaneo y operaciones no permitidas.
Tickets invalidos o invalidables	Tickets utilizados para validar restricciones de invalidación, compra, listado o escaneo.	Pruebas de contrato, API y scanner.
Tickets listados en reventa	Tickets marcados como disponibles en marketplace secundario.	Reventa manual guiada, cancelación y validación de marketplace.
Códigos QR válidos	Representaciones QR asociadas a tickets existentes y activos.	Scanner QR y pruebas E2E de verificación.
Códigos QR inválidos	QR falsos, expirados, stale, inexistentes o manipulados.	Casos negativos del scanner QR.
Eventos on-chain normalizados	Fixtures o estructuras simuladas de eventos emitidos por contratos Soroban.	Pruebas de integración del indexador y sincronización on-chain/off-chain.

6.5. Estados de ticket considerados

Las pruebas consideran diferentes estados del ticket para validar que el sistema permita o rechace operaciones según corresponda. La nomenclatura exacta puede variar de acuerdo con la implementación, pero conceptualmente se manejan los siguientes estados:

Estado	Descripción	Uso en pruebas
Activo / Disponible	Ticket válido que puede ser usado, comprado o listado según el flujo correspondiente.	Compra, reventa e inventario.

Listado en reventa	Ticket ofrecido en el marketplace secundario por su propietario.	Reventa y cancelación de reventa.
Vendido / Asignado	Ticket asociado a un comprador después de una compra primaria simulada o reventa.	Inventario y trazabilidad.
Usado	Ticket que ya fue verificado mediante scanner o redimido según el nivel evaluado.	Rechazo de doble escaneo y prevención de reutilización.
Invalidado	Ticket marcado como no operable por reglas del sistema o contrato.	Rechazo de compra, listado, escaneo o redención.
Cancelado de reventa	Ticket retirado del marketplace secundario por su propietario o por regla del sistema.	Validación de cancelación y rechazo de compra posterior.
Inexistente o manipulado	Identificador de ticket que no corresponde a un registro válido del sistema.	Casos negativos de API, scanner y seguridad.

6.6. Datos para pruebas de carga

Las pruebas de carga requieren datos estables y no destructivos que permitan ejecutar escenarios repetibles sobre endpoints REST. Para evitar alteración no controlada del estado del sistema, se priorizan endpoints de lectura y operaciones controladas.

Escenario	Datos requeridos	Observaciones
Lectura pública	Eventos existentes, marketplace o listado público de tickets.	No requiere autenticación ni modifica datos críticos.
Usuarios autenticados	Usuarios de prueba con credenciales válidas y tickets asociados.	Requiere tokens o credenciales configuradas mediante variables de entorno.
Operaciones críticas controladas	Usuario staff/admin, QR de prueba, ticket controlado o flujo de checkout simulado.	Debe evitar datos destructivos o reiniciar el estado antes de cada ejecución.

6.7. Consideraciones de seguridad sobre datos de prueba

Los datos utilizados durante las pruebas deben cumplir las siguientes condiciones:

- No deben incluir información personal sensible real.
- No deben utilizar activos con valor económico.
- No deben incluir claves privadas reales dentro del repositorio o del documento.
- Las wallets utilizadas deben corresponder exclusivamente a Stellar/Soroban Testnet.
- Las credenciales de prueba deben poder rotarse o reemplazarse sin afectar el sistema.
- Los códigos QR utilizados para pruebas negativas deben ser generados únicamente con fines de validación.
- Los datos destructivos o de cambio de estado deben aislarse para evitar afectar otros escenarios de prueba.

6.8. Criterios de preparación del ambiente

Antes de ejecutar las pruebas, se debe verificar que el ambiente cumpla con los siguientes criterios:

- El repositorio se encuentra en la rama y commit definidos para la ejecución de pruebas.
- Las dependencias del frontend, backend y contratos están instaladas correctamente.
- Las variables de entorno requeridas están configuradas.
- La base de datos está disponible y con migraciones aplicadas.
- Los usuarios, eventos, tickets y QR de prueba están creados o disponibles.
- Los contratos requeridos están compilados o desplegados según el tipo de prueba.
- La red configurada para blockchain corresponde a Stellar/Soroban Testnet.
- El backend y frontend se encuentran ejecutándose para pruebas E2E, smoke test o carga.

- Freightier Wallet está configurado con cuentas de Testnet para la prueba manual guiada de reventa.
- Los scripts de carga y guías manuales se encuentran actualizados en el repositorio.

7. Criterios de entrada y salida

Los criterios de entrada y salida establecen las condiciones mínimas que deben cumplirse antes de iniciar la ejecución de las pruebas y las condiciones necesarias para considerar que una prueba, grupo de pruebas o fase de validación ha finalizado satisfactoriamente. Estos criterios permiten controlar la ejecución del plan de pruebas, reducir ambigüedades y asegurar que los resultados obtenidos sean trazables y reproducibles.

Dado que Secure Ticket integra componentes frontend, backend, base de datos, contratos inteligentes y servicios externos de prueba, los criterios se definen tanto de forma general como por tipo de prueba.

7.1. Criterios generales de entrada

Antes de iniciar la ejecución formal de las pruebas, se deben cumplir las siguientes condiciones:

Criterio	Descripción	Estado
Versión congelada	Debe existir una rama, tag o commit definido para la ejecución de pruebas, evitando cambios no controlados durante la validación.	Pendiente
Repositorio disponible	El código fuente del frontend, backend, contratos y pruebas debe estar disponible en el repositorio oficial del proyecto.	Pendiente
Dependencias instaladas	Las dependencias requeridas para frontend, backend, contratos y herramientas de prueba deben estar instaladas correctamente.	Pendiente
Variables de entorno configuradas	Los archivos de configuración requeridos deben estar disponibles en el ambiente de prueba, sin exponer secretos reales.	Pendiente

Base de datos disponible	PostgreSQL debe estar disponible y con las migraciones aplicadas para ejecutar pruebas de API, integración y E2E.	Pendiente
Datos de prueba preparados	Usuarios, eventos, tickets, roles, códigos QR y wallets de Testnet deben estar disponibles según el tipo de prueba.	Pendiente
Contratos compilados	Los contratos inteligentes deben compilar correctamente antes de ejecutar pruebas de contrato o flujos blockchain.	Pendiente
Red blockchain configurada	Las pruebas blockchain deben estar configuradas exclusivamente contra Stellar/Soroban Testnet o entorno local de contrato.	Pendiente
Servicios ejecutándose	Para pruebas E2E, carga, smoke test o aceptación, el frontend y backend deben estar activos y accesibles.	Pendiente
Freighter configurado	Para la prueba manual de reventa, Freighter Wallet debe estar configurado con cuentas de Stellar Testnet y fondos de prueba suficientes.	Pendiente
Matriz de trazabilidad disponible	Debe existir una matriz base que relacione pruebas con requerimientos, casos de uso o atributos de calidad.	Pendiente

El estado de cada criterio será actualizado durante la fase de ejecución formal de pruebas.

7.2. Criterios generales de salida

Una vez ejecutadas las pruebas, se considerará que el proceso de validación cumple su salida general cuando se satisfagan las siguientes condiciones:

Criterio	Descripción	Estado
Pruebas ejecutadas	Las pruebas planificadas para el alcance definido deben haberse ejecutado o contar con justificación explícita en caso de no ejecución.	Pendiente
Resultados registrados	Cada prueba ejecutada debe contar con estado: aprobada, fallida, parcial, bloqueada o no aplica.	Pendiente

Fallos bloqueantes identificados	Los errores críticos encontrados durante la ejecución deben estar documentados y clasificados.	Pendiente
Fallos bloqueantes corregidos o justificados	Los defectos que impidan validar un flujo crítico deben corregirse o declararse como limitación del prototipo.	Pendiente
Evidencia asociada	Las pruebas ejecutadas deben contar con evidencia correspondiente, como logs, resultados de consola, capturas, reportes, tx hashes o formularios.	Pendiente
Trazabilidad actualizada	La matriz de trazabilidad debe reflejar el estado real de implementación y ejecución de cada prueba.	Pendiente
Limitaciones documentadas	Las coberturas parciales, exclusiones y decisiones de alcance deben estar explícitamente registradas.	Pendiente
Conclusión de calidad	Debe existir una conclusión sobre el estado de calidad del prototipo con base en los resultados obtenidos.	Pendiente

7.3. Criterios de entrada por tipo de prueba

Además de los criterios generales, cada tipo de prueba requiere condiciones específicas antes de su ejecución.

Tipo de prueba	Criterios de entrada
Pruebas de contratos	Los contratos deben compilar correctamente; el entorno Rust/Soroban debe estar configurado; los casos de prueba deben estar implementados en el repositorio.
Pruebas backend/API	El backend debe tener dependencias instaladas; la base de datos de prueba debe estar disponible; las variables de entorno deben estar configuradas; los usuarios y roles de prueba deben existir o ser creados por fixtures.
Pruebas de integración	La base de datos debe estar migrada; los fixtures o eventos normalizados deben estar disponibles; el servicio de indexación o módulo equivalente debe poder ejecutarse en ambiente de prueba.

Pruebas frontend/E2E	El frontend y backend deben estar ejecutándose; los usuarios de prueba deben existir; los datos mínimos de eventos y tickets deben estar disponibles; la herramienta E2E debe estar configurada.
Prueba manual de reventa Testnet	Freighter debe estar instalado; las wallets de Testnet deben estar configuradas; los usuarios deben estar vinculados a wallets; el ticket a revender debe existir; los contratos y backend deben apuntar a Testnet.
Pruebas del scanner QR	Deben existir tickets válidos, usados, inválidos o inexistentes según el caso; los usuarios STAFF y ADMIN deben estar disponibles; los QR de prueba deben estar generados.
Pruebas de carga	El backend debe estar disponible; los endpoints definidos deben responder correctamente en condiciones normales; las credenciales o tokens de prueba deben estar configurados; los datos usados deben ser estables.
Smoke test de despliegue	Las URLs de frontend y backend deben estar disponibles; el commit desplegado debe estar identificado; los usuarios y datos mínimos deben existir.
Pruebas de aceptación	La guía de tareas debe estar definida; el formulario debe estar preparado; los usuarios o stakeholders deben estar seleccionados; el ambiente debe estar estable para ejecutar las tareas.
Trazabilidad	La matriz debe contener los identificadores de pruebas, su tipo, nivel y relación con requerimientos o casos de uso.

7.4. Criterios de salida por tipo de prueba

Cada grupo de pruebas se considera finalizado cuando cumple los siguientes criterios de salida:

Tipo de prueba	Criterios de salida
Pruebas de contratos	Todas las pruebas ejecutadas deben pasar o, en caso de falla, el defecto debe estar documentado. Deben quedar cubiertos los casos críticos de emisión, reventa, cancelación, invalidación, redención y negativos relevantes.

Pruebas backend/API	Los endpoints críticos deben responder según lo esperado; las reglas de negocio, autorización, scanner y validaciones de estado deben aprobarse sin fallos bloqueantes.
Pruebas de integración	Los eventos o estados procesados deben reflejarse correctamente en PostgreSQL; no debe haber duplicaciones ni inconsistencias visibles en los datos evaluados.
Pruebas frontend/E2E	Los flujos principales deben completarse desde la interfaz o mostrar errores controlados cuando corresponda. Los errores visuales o funcionales críticos deben quedar registrados.
Prueba manual de reventa Testnet	La guía debe ejecutarse completamente o registrar el punto exacto de bloqueo. Debe quedar claro si hubo cambio de propietario, actualización de estado y transacción en Testnet.
Pruebas del scanner QR	El scanner debe aceptar QR válidos y rechazar QR inválidos, usados, expirados, inexistentes o ejecutados por usuarios no autorizados.
Pruebas de carga	Los escenarios deben completar su ejecución y reportar métricas de error rate, promedio, máximo y percentil 95. Los resultados deben compararse con los umbrales definidos.
Smoke test de despliegue	Los servicios principales deben estar disponibles y permitir ejecutar los flujos mínimos definidos para el ambiente desplegado.
Pruebas de aceptación	Los participantes deben completar las tareas asignadas o registrar los bloqueos encontrados. Los formularios deben consolidarse para análisis.
Trazabilidad	La matriz debe actualizarse con el estado real de cada prueba y su relación con requerimientos, casos de uso o atributos de calidad.

7.5. Estados de resultado de prueba

Para unificar el registro de resultados, cada prueba será clasificada con uno de los siguientes estados:

Estado	Descripción
--------	-------------

Aprobada	La prueba se ejecutó y el resultado obtenido coincide con el resultado esperado.
Fallida	La prueba se ejecutó, pero el resultado obtenido no coincide con el resultado esperado.
Parcial	La prueba se ejecutó o implementó parcialmente, pero no cubre la totalidad del flujo o depende de una limitación del prototipo.
Bloqueada	La prueba no pudo ejecutarse por ausencia de ambiente, datos, credenciales, dependencia externa o defecto bloqueante.
No ejecutada	La prueba está definida o implementada, pero aún no se ha ejecutado formalmente.
No aplica	La prueba no corresponde al alcance del prototipo o requiere capacidades fuera del alcance académico, como mainnet, pagos reales o infraestructura productiva.

7.6. Criterio de suspensión y reanudación

Una prueba o grupo de pruebas podrá suspenderse temporalmente cuando se presente alguna de las siguientes condiciones:

- El ambiente requerido no se encuentra disponible.
- La base de datos no está migrada o no contiene los datos mínimos.
- Las variables de entorno no están configuradas correctamente.
- El frontend o backend no inicia correctamente.
- Freightier no está configurado para Stellar/Soroban Testnet en la prueba manual de reventa.
- Una dependencia externa de prueba, como Stellar/Soroban Testnet, no responde adecuadamente.
- Se identifica un defecto bloqueante que impide continuar con el flujo evaluado.

La ejecución podrá reanudarse cuando la condición que generó la suspensión sea corregida, documentada o reemplazada por un escenario equivalente dentro del alcance del prototipo.

8. Casos de prueba

Esta sección presenta los casos de prueba definidos para validar los componentes y flujos principales de Secure Ticket. Los casos se organizan por tipo de prueba y se relacionan con los elementos críticos del sistema: contratos inteligentes, backend/API, frontend, scanner QR, reventa en Testnet, sincronización on-chain/off-chain, seguridad, carga, despliegue y aceptación de usuario.

Los casos descritos en esta sección corresponden a pruebas implementadas, pruebas automatizadas listas para ejecución o guías manuales definidas para flujos que no se automatizan por restricciones técnicas, como la interacción con Freighter Wallet. El estado final de cada caso será actualizado después de la ejecución formal de las pruebas.

8.1. Formato de los casos de prueba

Para mantener consistencia en el registro de pruebas, cada caso se describe mediante los siguientes campos:

Campo	Descripción
ID	Identificador único del caso de prueba.
Nombre	Nombre corto del caso de prueba.
Tipo	Clasificación de la prueba: contrato, API, integración, E2E, carga, manual guiada, aceptación o documental.
Precondiciones	Condiciones necesarias antes de ejecutar la prueba.
Pasos resumidos	Secuencia general de acciones requeridas para ejecutar el caso.
Resultado esperado	Comportamiento que debe presentar el sistema para considerar la prueba aprobada.
Estado	Estado de ejecución de la prueba: pendiente, aprobada, fallida, parcial, bloqueada o no aplica.

8.2. Casos de prueba de contratos inteligentes

Los casos de prueba de contratos inteligentes validan la lógica on-chain implementada en Soroban. Estas pruebas se ejecutan principalmente mediante **cargo test** y permiten verificar que las reglas críticas del ciclo de vida del ticket funcionen correctamente.

ID	Caso	Precondiciones	Resultado esperado	Estado
CONTRACT- ISSUE-01	Emisión de boleto	Contrato inicializado y datos válidos de evento/ticket.	El contrato crea un boleto único asociado al evento y al propietario correspondiente.	Pendiente
CONTRACT- RESALE-01	Reventa de boleto	Existe un boleto activo y propietario autorizado.	El contrato permite ejecutar la reventa bajo las reglas definidas y actualiza la propiedad del ticket.	Pendiente
CONTRACT- BURN- REMINT-01	Burn/remint o versionamiento de ticket	Existe un ticket que será transferido en reventa.	La versión anterior queda no operable y se refleja una nueva versión o estado equivalente del ticket.	Pendiente
CONTRACT- COMMISSION- 01	Distribución de comisiones	Existe una operación de reventa con precio y reglas de comisión definidas.	El contrato calcula y distribuye las comisiones según la lógica establecida.	Pendiente
CONTRACT- RESALE- CANCEL-01	Cancelación de reventa	Existe un ticket listado para reventa por su propietario.	El propietario puede cancelar la venta y el ticket deja de estar disponible en reventa.	Pendiente
CONTRACT- INVALIDATE- 01	Invalidación de boleto	Existe un ticket activo o listado.	El boleto invalidado no puede venderse, listarse, comprarse ni redimirse posteriormente.	Pendiente
CONTRACT- REDEEM-01	Redención on-chain	Existe un ticket válido y un verificador autorizado.	El contrato permite la redención por verificador autorizado y rechaza redenciones no autorizadas o repetidas.	Pendiente

CONTRACT-NEG-01	Casos negativos de contrato	Existen entradas inválidas: ticket inexistente, usuario no autorizado, boleto usado o invalidado.	El contrato rechaza las operaciones inválidas sin alterar indebidamente el estado.	Pendiente
-----------------	-----------------------------	---	--	-----------

8.3. Casos de prueba backend/API

Los casos de prueba backend/API verifican los endpoints, reglas de negocio, seguridad funcional y validaciones realizadas del lado del servidor. Estas pruebas se ejecutan mediante el framework de pruebas del backend y Supertest o herramienta equivalente.

ID	Caso	Precondiciones	Resultado esperado	Estado
API-AUTH-01	Acceso sin token	Endpoint protegido disponible.	El sistema rechaza la solicitud con código 401 o equivalente.	Pendiente
API-AUTH-02	Token inválido o vencido	Se utiliza un token inválido, mal formado o expirado.	El sistema rechaza la solicitud y no ejecuta la operación.	Pendiente
SEC-ROLE-01	Validación de roles	Usuarios con roles CUSTOMER, STAFF y ADMIN disponibles.	Cada rol solo puede ejecutar las operaciones permitidas.	Pendiente
API-TICKET-OWNER-01	Operación sobre ticket ajeno	Usuario autenticado intenta operar un ticket que no le pertenece.	El backend rechaza la operación.	Pendiente
API-REQUEST-MANIP-01	Manipulación de request	Solicitud con precio, estado o identificador manipulado.	El backend valida la información y rechaza operaciones inválidas.	Pendiente
API-SCAN-NEG-01	Scanner QR con negativos	Existen QR válidos, inválidos, usados, expirados e inexistentes.	El scanner acepta QR válidos y rechaza casos inválidos o no autorizados.	Pendiente

API-TICKET-INVALIDATE-01	Invalidación de ticket	Existe ticket activo o invalidable según la implementación.	El sistema bloquea operaciones posteriores sobre tickets inválidos o invalidados.	Parcial
API-RESALE-CANCEL-01	Cancelación de reventa	Existe ticket listado para reventa.	La cancelación retira el ticket del marketplace y evita compras posteriores.	Pendiente
API-CHECKIN-01	Registro de check-in	Ticket válido y usuario STAFF/ADMIN.	El sistema registra el check-in y evita duplicación en escaneos posteriores.	Pendiente

8.4. Casos de prueba de integración

Los casos de integración validan la interacción entre backend, base de datos, indexador y eventos asociados a la lógica on-chain. En esta categoría se aceptan eventos normalizados o fixtures cuando la prueba no depende directamente de una transacción real en Testnet.

ID	Caso	Precondiciones	Resultado esperado	Estado
INT-SYNC-01	Sincronización de eventos	Base de datos disponible y eventos normalizados definidos.	Los eventos procesados actualizan correctamente PostgreSQL.	Pendiente
INT-SYNC-02	Evento de boleto creado	Evento de creación disponible.	Se crea o actualiza el registro del ticket con los campos esperados.	Pendiente
INT-SYNC-03	Evento de boleto listado	Evento de listado disponible.	El ticket queda marcado como disponible para reventa.	Pendiente
INT-SYNC-04	Evento de venta cancelada	Evento de cancelación disponible.	El ticket deja de estar listado para reventa.	Pendiente

INT-SYNC-05	Evento de boleto re- vendido	Evento de reventa dis- ponible.	Se actualiza propie- tario, versión y esta- do del ticket.	Pendiente
INT-SYNC-06	Evento de boleto re- dimido	Evento de redención disponible.	El ticket queda mar- cado como usado o redimido según el modelo de datos.	Pendiente
INT-SYNC-07	Evento de boleto in- validado	Evento de invalidación disponible.	El ticket queda blo- queado para opera- ciones posteriores.	Pendiente
INT-SYNC-08	Evento duplicado	Se procesa dos veces el mismo evento.	El sistema mantie- ne idempotencia y no duplica registros.	Pendiente
INT-SYNC-09	Evento desconocido o mal formado	Evento no reconocido o con estructura inválida.	El sistema maneja el evento de forma con- trolada sin romper el proceso.	Pendiente

8.5. Casos de prueba frontend y end-to-end

Los casos frontend/E2E validan que los usuarios puedan completar flujos representativos desde la interfaz web del prototipo.

ID	Caso	Precondiciones	Resultado espera- do	Estado
E2E-BUY-01	Compra primaria si- mulada	Usuario CUSTOMER, evento disponible y bac- kend activo.	El usuario completa el checkout simulado y visualiza el ticket en inventario.	Pendiente
E2E-BUY-02	Usuario no autenti- cado intenta comprar	Usuario sin sesión acti- va.	El sistema bloquea la compra o redirige al inicio de sesión.	Pendiente

E2E-BUY-03	Error de checkout simulado	Condición de error controlada en checkout.	El sistema muestra un error comprensible y no deja estado inconsistente.	Pendiente
E2E-INVENTORY-01	Consulta de inventario	Usuario autenticado con tickets asociados.	El inventario muestra los tickets correspondientes al usuario.	Pendiente
E2E-SCAN-01	Scanner QR desde UI	Usuario STAFF/ADMIN y QR válido disponible.	El scanner muestra éxito para QR válido y registra el uso del ticket.	Pendiente
E2E-SCAN-02	Doble escaneo desde UI	Ticket previamente escaneado.	El sistema rechaza el segundo escaneo y muestra mensaje de error.	Pendiente
E2E-SCAN-03	QR inválido desde UI	QR inválido o manipulado.	El sistema rechaza el QR y muestra un mensaje de error comprensible.	Pendiente
E2E-SCAN-04	CUSTOMER intenta acceder al scanner	Usuario con rol CUSTOMER.	El sistema bloquea el acceso a la funcionalidad de scanner.	Pendiente

8.6. Caso de prueba manual de reventa en Testnet

La reventa con Freighter se valida mediante una guía manual debido a la fragilidad de automatizar extensiones de wallet. Este caso representa el flujo central de la propuesta de Secure Ticket.

Campo	Descripción
ID	E2E-REV-01
Nombre	Reventa manual guiada en Stellar/Soroban Testnet con Freighter.
Tipo	Manual guiada / Integración blockchain.

Precondiciones	Usuario A y Usuario B con cuentas y wallets de Testnet configuradas; Usuario A posee un ticket válido; contratos y backend apuntan a Stellar/Soroban Testnet; Freightier está instalado y configurado; PostgreSQL contiene los datos mínimos.
Pasos resumidos	Usuario A inicia sesión, conecta Freightier, lista un ticket propio para reventa; Usuario B inicia sesión, conecta Freightier y compra el ticket listado; se valida la firma, transacción en Testnet, cambio de propietario, actualización de inventario y consistencia con PostgreSQL.
Casos negativos	Compra del propio ticket, listado de ticket ajeno, compra de ticket cancelado, rechazo de firma en Freightier y wallet firmante distinta al usuario autenticado cuando aplique.
Resultado esperado	La reventa se completa en Stellar/Soroban Testnet, el ticket cambia de propietario, el marketplace e inventario se actualizan y los casos negativos son rechazados.
Estado	Pendiente de ejecución formal.

8.7. Casos de prueba de carga

Las pruebas de carga se organizan en tres escenarios: lectura pública, usuarios autenticados y operaciones críticas controladas. Estas pruebas se ejecutan sobre endpoints REST y no sobre mainnet, Freightier ni transacciones Soroban masivas.

ID	Escenario	Precondiciones	Resultado esperado	Estado
LOAD-PUBLIC-01	Lectura pública	Backend disponible, eventos y marketplace configurados.	Los endpoints públicos responden bajo carga moderada con error rate y p95 dentro del umbral.	Pendiente
LOAD-AUTH-01	Usuarios autenticados	Usuarios de prueba y credenciales válidas disponibles.	Login, inventario y endpoints autenticados responden dentro de los umbrales definidos.	Pendiente

LOAD-CRITICAL-01	Operaciones críticas controladas	Usuario STAFF/ADMIN, QR o datos controlados y backend estable.	Scanner, checkout simulado u operaciones seleccionadas responden sin errores significativos.	Pendiente
------------------	----------------------------------	--	--	-----------

8.8. Casos de prueba de smoke test de despliegue

El smoke test valida que el prototipo funcione de forma básica en el ambiente desplegado.

ID	Caso	Precondiciones	Resultado esperado	Estado
SMOKE-DEPLOY-01	Frontend disponible	URL de frontend desplegada.	La aplicación carga correctamente en navegador.	Pendiente
SMOKE-DEPLOY-02	Backend disponible	URL de backend desplegada.	El endpoint de salud responde correctamente.	Pendiente
SMOKE-DEPLOY-03	Login en despliegue	Usuario de prueba disponible.	El usuario puede iniciar sesión.	Pendiente
SMOKE-DEPLOY-04	Consulta de eventos y marketplace	Datos públicos disponibles.	Los eventos y marketplace cargan correctamente.	Pendiente
SMOKE-DEPLOY-05	Inventario y scanner	Usuario con tickets y usuario STAFF/ADMIN disponibles.	El inventario y scanner funcionan según el rol correspondiente.	Pendiente

8.9. Casos de prueba de aceptación de usuario

Las pruebas de aceptación se orientan a validar que usuarios o stakeholders puedan completar tareas representativas del prototipo y comprender los estados principales del ticket.

ID	Perfil	Tarea	Resultado esperado	Estado
----	--------	-------	--------------------	--------

UAT-01	Comprador	Iniciar sesión, consultar eventos, comprar ticket y revisar inventario.	El usuario completa la compra simulada y comprende el estado del ticket.	Pendiente
UAT-02	Vendedor/revendedor	Seleccionar ticket propio y listarlo para reventa.	El usuario comprende el proceso de publicación del ticket en marketplace.	Pendiente
UAT-03	Comprador de reventa	Consultar marketplace y comprar un ticket de reventa si el flujo está disponible.	El usuario comprende el flujo de reventa y el cambio de propiedad.	Pendiente
UAT-04	Staff/verificador	Escanear QR válido e intentar doble escaneo.	El usuario comprende el resultado del scanner y la prevención de doble uso.	Pendiente
UAT-05	Admin/organizador	Revisar eventos, tickets o estados administrativos disponibles.	El usuario identifica la información operativa relevante del sistema.	Pendiente

8.10. Casos documentales de trazabilidad

La trazabilidad relaciona pruebas con requerimientos, casos de uso y atributos de calidad.

ID	Caso	Precondiciones	Resultado esperado	Estado
TRACE-01	Matriz de trazabilidad	Requerimientos, casos de uso y pruebas identificadas.	Cada prueba crítica se asocia con al menos un requerimiento, caso de uso o atributo de calidad.	Pendiente

TRACE-02	Cobertura parcial	Funcionalidades con cobertura parcial identificadas.	Las limitaciones quedan documentadas sin afirmar capacidades fuera del alcance.	Pendiente
TRACE-03	Exclusiones por alcance	Elementos fuera del alcance definidos.	Mainnet, pagos reales, carga blockchain masiva y automatización completa de Freightler quedan excluidos explícitamente.	Pendiente

8.11. Resumen de cobertura de casos

La siguiente tabla resume la cobertura esperada por categoría de prueba.

Categoría	Casos principales	Cobertura esperada
Contratos inteligentes	CONTRACT-ISSUE, CONTRACT-RESALE, CONTRACT-REDEEM, CONTRACT-INVALIDATE	Lógica on-chain de emisión, reventa, redención, invalidación y negativos.
Backend/API	API-SCAN, SEC-ROLE, API-RESALE-CANCEL, API-TICKET-INVALIDATE	Reglas de negocio, seguridad funcional, scanner, cancelación e invalidación parcial.
Integración	INT-SYNC	Sincronización entre eventos normalizados y PostgreSQL.
Frontend/E2E	E2E-BUY, E2E-SCAN	Compra primaria simulada, inventario y scanner desde interfaz.

Reventa Testnet	E2E-REV-01	Flujo manual guiado de reventa con Freightier sobre Stellar/Soroban Testnet.
Carga	LOAD-PUBLIC, LOAD-AUTH, LOAD-CRITICAL	Rendimiento básico sobre endpoints REST representativos.
Despliegue	SMOKE-DEPLOY	Disponibilidad mínima de frontend, backend y flujos básicos.
Aceptación	UAT	Validación académica con usuarios o stakeholders.
Trazabilidad	TRACE	Relación entre pruebas, requerimientos, casos de uso y limitaciones.

9. Resultados y análisis

9.1. Resumen general de ejecución

Tipo de prueba	Ejecutadas	Exitosas	Fallidas	No ejecutadas
Unitarias	[N]	[N]	[N]	[N]
Sistema	[N]	[N]	[N]	[N]
Carga	[N]	[N]	[N]	[N]
Aceptación de usuario	[N]	[N]	[N]	[N]
Configurabilidad	[N]	[N]	[N]	[N]

9.2. Resultados de pruebas unitarias

[Describir resultados por servicio: Users, Products, Orders y Reviews. Incluir cobertura, errores encontrados y correcciones realizadas.]

9.3. Resultados de pruebas de sistema

[Describir resultados por flujo: creación/edición de productos, compra, seguimiento en tiempo real y seguimiento por estados.]

9.4. Resultados de pruebas de carga

Endpoint/Controlador	Tiempo promedio	Throughput	Tasa de error	Conclusión
<i>[Endpoint]</i>	<i>[ms]</i>	<i>[req/s]</i>	<i>[%]</i>	<i>[Aprobado/Fallido]</i>

9.5. Resultados de aceptación de usuario

[Incluir el análisis de encuestas, comentarios de usuarios, porcentajes de aceptación y observaciones relevantes.]

9.6. Resultados de configurabilidad

[Describir el comportamiento del derivador frente a producto mínimo, producto máximo, producto no válido y producto con restricciones aplicadas.]

9.7. Análisis de defectos

ID defec-to	Descripción	Severidad	Estado	Acción tomada
DEF-01	<i>[Descripción del defec-to]</i>	<i>[Alta/Media/Baja]</i>	<i>[Abierto/Cerrado]</i>	<i>[Acción]</i>

10. Riesgos y mitigaciones

ID	Riesgo	Impacto	Mitigación
----	--------	---------	------------

R-01	Ambiente de pruebas inestable.	Alto	Separar ambiente de pruebas y documentar configuración.
R-02	Datos de prueba insuficientes.	Medio	Preparar datos válidos, inválidos y de borde antes de ejecutar pruebas.
R-03	Pruebas de carga no representativas.	Medio	Definir escenarios de carga baja y alta con parámetros claros.
R-04	Baja participación en pruebas de aceptación.	Medio	Definir tareas cortas, claras y medibles para los usuarios participantes.
R-05	Configuraciones SPL no alineadas con el árbol de características.	Alto	Validar cada configuración contra FeatureIDE y registrar evidencia.

11. Matriz de trazabilidad

Requerimiento	Descripción	Caso(s) de prueba asociado(s)	Estado de validación
REQ-01	<i>[Descripción del requerimiento]</i>	<i>[USR-01, PS-01, etc.]</i>	<i>[Validado/Pendiente/Fallido]</i>
REQ-02	<i>[Descripción del requerimiento]</i>	<i>[Casos asociados]</i>	<i>[Estado]</i>
REQ-03	<i>[Descripción del requerimiento]</i>	<i>[Casos asociados]</i>	<i>[Estado]</i>

12. Conclusiones

[Redactar las conclusiones principales del proceso de pruebas. Indicar si el sistema cumple los criterios de aceptación, cuáles fueron los principales hallazgos y qué aspectos requieren mejora.]

12.1. Trabajo futuro

[Indicar pruebas o mejoras futuras: automatización E2E, pruebas de seguridad, pruebas de usabilidad más formales, pruebas en producción controlada, etc.]

13. Anexos

13.1. Anexo A: Evidencias de pruebas unitarias

[Agregar capturas, reportes de cobertura o enlaces a resultados.]

13.2. Anexo B: Evidencias de pruebas de sistema

[Agregar capturas de pantalla, colecciones de Postman o resultados manuales.]

13.3. Anexo C: Evidencias de pruebas de carga

[Agregar reportes de JMeter, tablas de métricas y gráficos.]

13.4. Anexo D: Formularios de aceptación

[Agregar preguntas, respuestas consolidadas y resultados de usuarios.]

13.5. Anexo E: Evidencias de configurabilidad

[Agregar árbol de características, configuraciones generadas y capturas del derivador.]